

Manual

Get the code and data

1. clone the repo

git clone <https://github.com/lancercat/Moose.git>

2. Get data from

1. <https://www.kaggle.com/datasets/vsdf2898kaggle/osocrtraining?rvi=1>

and

2. <https://www.kaggle.com/datasets/object300/mose-extra>

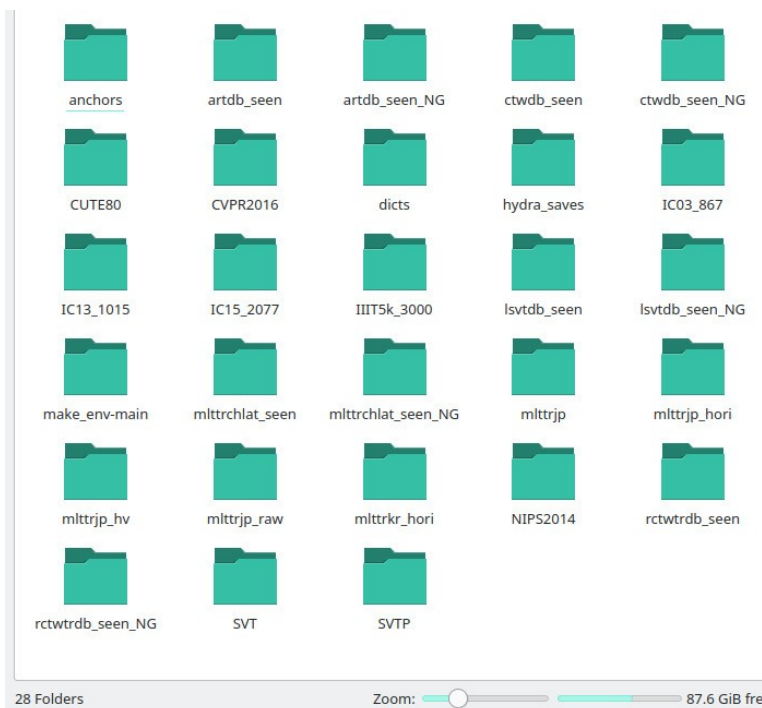
Unzip 1 first, then override it with 2. you should get around 25 folders

E: You will miss `mlttrjp` and `mlttrjp_raw`. They are files generated while I was testing the data builder.

`mlttrjp_raw` is whatever annotated as japan in the `mltdataset`.

`Mlttrjp` is `mlttrjp_hv`. It is a filtered version of `mlttrjp_raw`, by removing samples include symbols and korean characters.

If you have one or two missing or something fails to run, please email me



3. make a folder named anchors in the data dir, now you have 28

```
mkdir -p ~/ssddata/anchors/
```

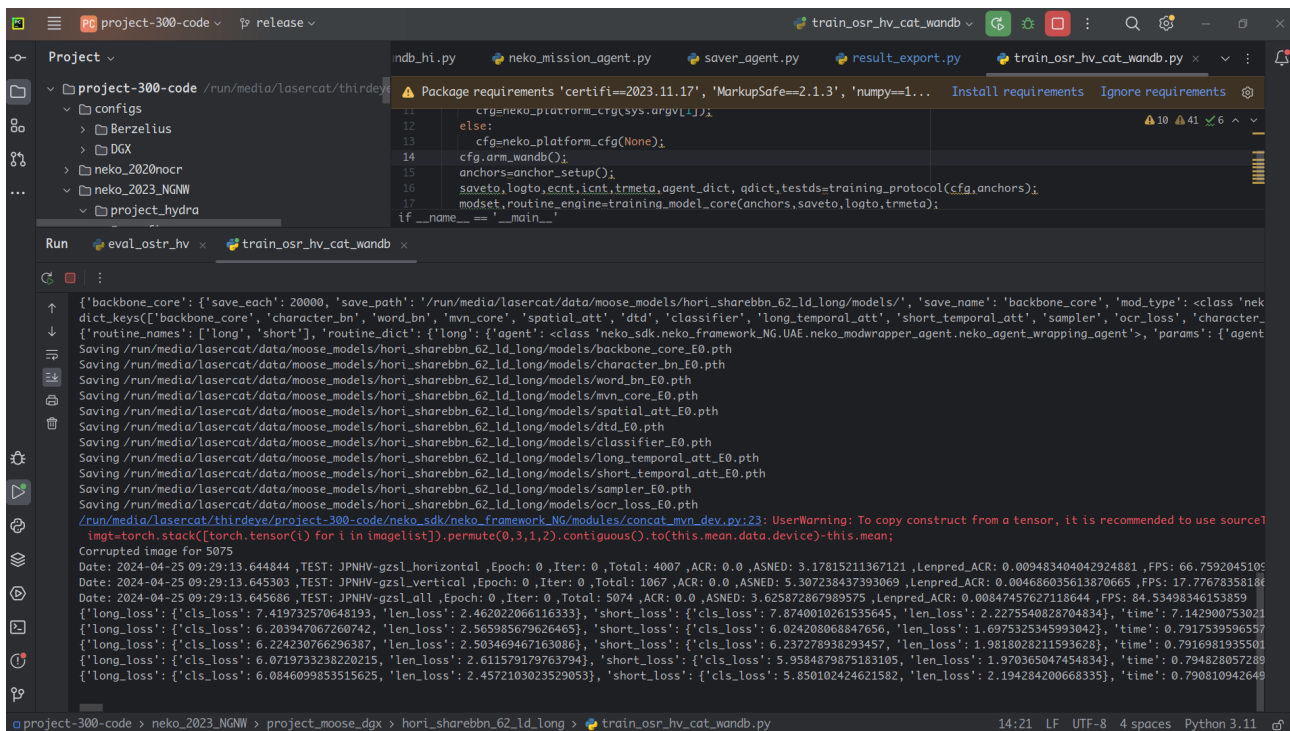
4. Put a “hydra_saves” folder in your home directory.

```
mkdir -p ~/hydra_saves
```

5. Try training a moose from pycharm. If it works, it works.

Allow it some time for the first runs. It will build index files if it fails to load them from the disk.

It will also launch a testing dry run so we don't fail midway.



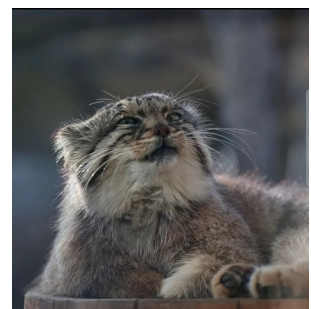
```
Package requirements 'certifi==2023.11.17', 'MarkupSafe==2.1.3', 'numpy==1... Install requirements Ignore requirements
```

```
11: cfg=neko_platform_cfg(sys.argv[1:])
12: else:
13:     cfg=neko_platform_cfg(None)
14:     cfg.arm_wandb();
15:     anchors=anchor_setup();
16:     saveto, logto, cnt, icnt, trmeta, agent_dict, adict, testds=training_protocol(cfg, anchors);
17:     modset, routine_engine=training_model_core(anchors, saveto, logto, trmeta);
if __name__ == '__main__':
```

```
{'backbone_core': {'save_each': 20000, 'save_path': '/run/media/lasercat/data/moose_models/hori_sharebnn_62_ld_long/models/', 'save_name': 'backbone_core', 'mod_type': <class 'neko_sdk.neko_framework_ng.modules.concat_mvn_dev.py:23: UserWarning: To copy construct from a tensor, it is recommended to use source
img=torch.stack([torch.tensor(l) for l in imagelist]).permute(0,3,1,2).contiguous().to(this.mean.data.device)-this.mean;
Corrupted image for 5075
Date: 2024-04-25 09:29:13.644844, TEST: JPNHV-gzsl_horizontal, Epoch: 0, Iter: 0, Total: 4007, ACR: 0.0, ASNED: 3.17815211367121, Lenpred_ACR: 0.009483404042924881, FPS: 66.7592045105
Date: 2024-04-25 09:29:13.645303, TEST: JPNHV-gzsl_vertical, Epoch: 0, Iter: 0, Total: 1067, ACR: 0.0, ASNED: 5.307238437393069, Lenpred_ACR: 0.004686035613870665, FPS: 17.77678358186
Date: 2024-04-25 09:29:13.645686, TEST: JPNHV-gzsl_all, Epoch: 0, Iter: 0, Total: 5074, ACR: 0.0, ASNED: 3.625872867989575, Lenpred_ACR: 0.00847457627118644, FPS: 84.53498346153859
{'long_loss': {'cls_loss': 7.419732570648193, 'len_loss': 2.462022066116333}, 'short_loss': {'cls_loss': 7.8740010261535645, 'len_loss': 2.2275540828704834}, 'time': 7.142900753021
{'long_loss': {'cls_loss': 6.203947067260742, 'len_loss': 2.565985679626465}, 'short_loss': {'cls_loss': 6.024208068847656, 'len_loss': 1.6975325345993042}, 'time': 0.7917539596557
{'long_loss': {'cls_loss': 6.224230766296387, 'len_loss': 2.503469467163086}, 'short_loss': {'cls_loss': 6.237278938293457, 'len_loss': 1.9818028211593628}, 'time': 0.7916981935501
{'long_loss': {'cls_loss': 6.0719733238220215, 'len_loss': 2.611579179763794}, 'short_loss': {'cls_loss': 5.9584879875183105, 'len_loss': 1.970365047454834}, 'time': 0.794828057289
{'long_loss': {'cls_loss': 6.0846099853515625, 'len_loss': 2.4572103023529053}, 'short_loss': {'cls_loss': 5.850102424621582, 'len_loss': 2.194284200668335}, 'time': 0.790810942649
```

OK, that's all. If it fails, open an issue or mail lasercat@gmx.us

Enjoy.



Reproducing Ablative Studies

1. Download the logs

<https://www.kaggle.com/datasets/object300/log-moose-release>

```
[lasercat@MEOWS-Gamarket project_moose]$ pwd
/run/media/lasercat/thirdeye/moose_logs_models/all/project_moose
[lasercat@MEOWS-Gamarket project_moose]$ ls
hori_sharebbn_62_ld_long_10434818_2024-02-07-12-06-42  sharebbn_62_ld_long_10434821_2024-02-07-12-07-17
hori_sharebbn_s_05_ld_long_10434820_2024-02-07-12-07-07  sharebbn_62RS_ld_long_10434822_2024-02-07-12-07-27
shareall_62_ld_long_10452003_2024-02-09-08-38-28  sharenone_62_ld_long_10452004_2024-02-09-08-38-33
sharebbn_05_ld_long_10436205_2024-02-07-13-32-28  yukon_sharebbn_62_ld_long_XL_10434825_2024-02-07-12-07-49
[lasercat@MEOWS-Gamarket project_moose]$
```

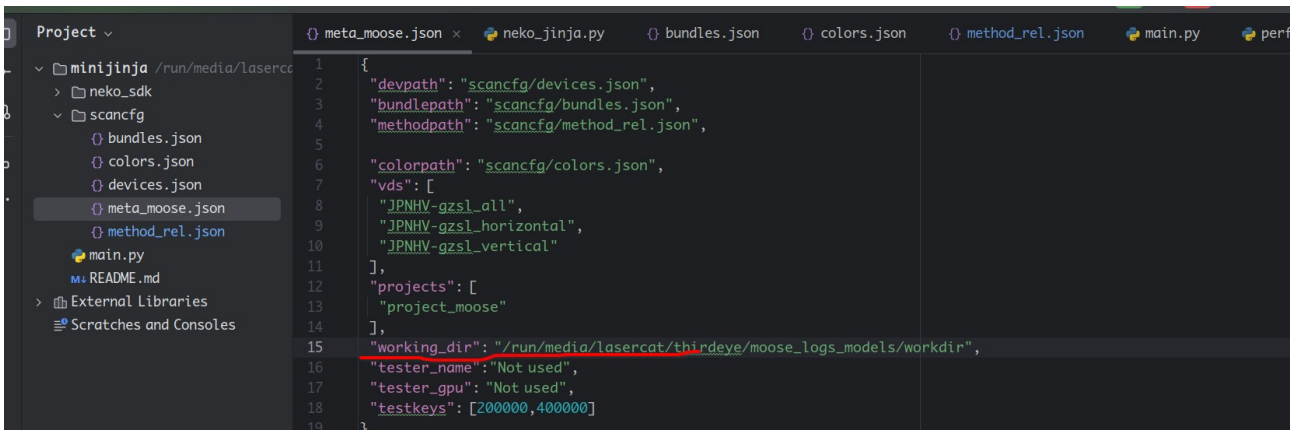
2 Download the visualizer:

git clone <https://github.com/lancercat/minijinja.git>

Set your working path in scancfg/metacfg.json

WARNING: I forgot if this thing will or will not drop your workingdir,

check yourself, put it in a sandbox, or make sure the workingdir is set right.



```
1 {
2   "devpath": "scancfg/devices.json",
3   "bundlepath": "scancfg/bundles.json",
4   "methodpath": "scancfg/method_rel.json",
5
6   "colorpath": "scancfg/colors.json",
7   "vds": [
8     "JPNHV-gzsl_all",
9     "JPNHV-gzsl_horizontal",
10    "JPNHV-gzsl_vertical"
11  ],
12  "projects": [
13    "project_moose"
14  ],
15  "working_dir": "/run/media/lasercat/thirdeye/moose_logs_models/workdir",
16  "tester_name": "Not used",
17  "tester_gpu": "Not used",
18  "testkeys": [200000, 400000]
19 }
```

Also set your device info in devices.json

a) Specifically, set your username, sshkey (that can allow you access localhost, or wherever you store the downloaded logs)

b) Set root to where your downloaded logs are (pwd in step 1)

```

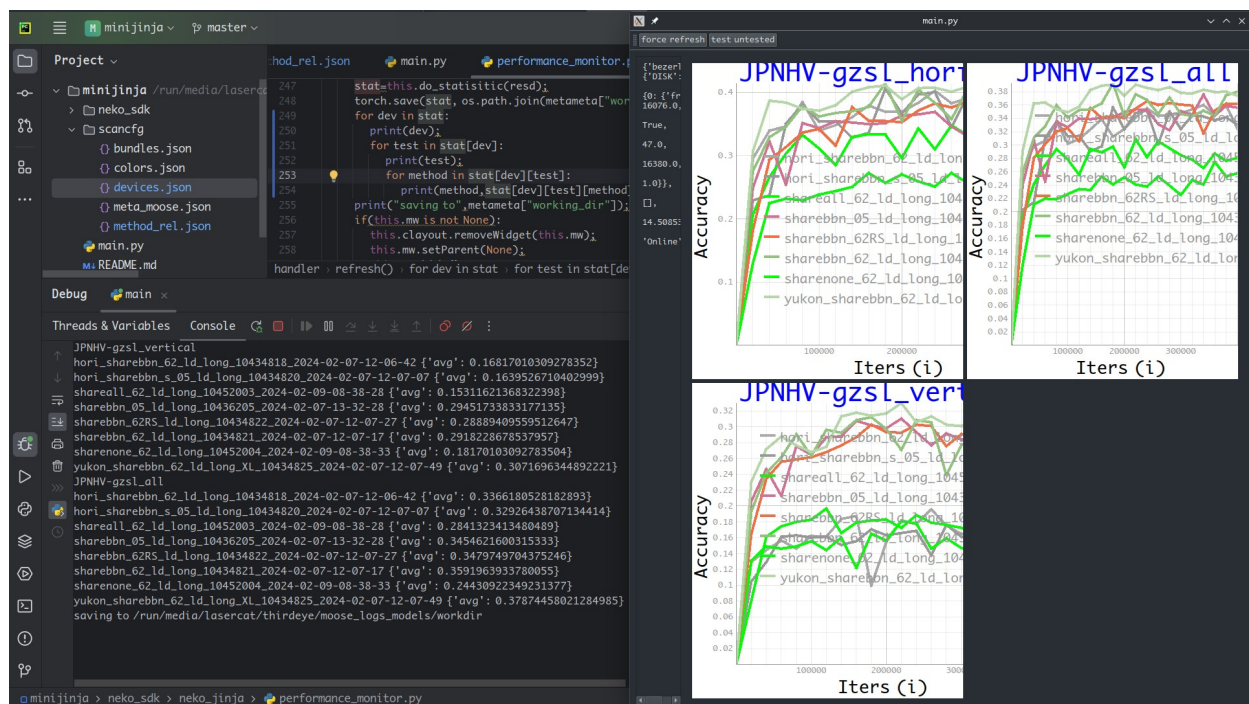
od_rel.json  main.py  performance_monitor.py  collect_res_osocrNG.py

{
  "bezerlius": {
    "ltype": "solid",
    "name": "bezerlius",
    "username": "lasercat",
    "key": "/home/lasercat/.ssh/id_rsa",
    "root": "/run/media/lasercat/thirdeye/moose_logs_models/all/",
    "port": "22",
    "addr": "localhost"
  }
}

```

Trivia: This library is designed by me to periodically monitor, grab logs from, and issue commands to a fleet of servers located in different places, thus it uses ssh/rsync/paramiko.

3. Go run the script main.py You are golden if you see the curves we presented in SSBA.



Note for `sharebbn_62RS_ld_long_10434822_2024-02-07-12-07-27` the horizontal/vertical stats DO NOT make sense, because the vertical images with aspect ratio >2 are rotated without the logger knowing, so the numbers come from a different split of all test images. That's why we only report overall accuracy in Table 3.

4. Now let's match the statistical data to the logs in the tables in the paper.

Table2, Row 1

Horizontal	Dropped	Horizontal	32.93	16.4	37.33
hori_sharebbn_s_05_ld_long_10434820_2024-02-07-12-07-07 {'avg': 0.32926438707134414}					
hori_sharebbn_s_05_ld_long_10434820_2024-02-07-12-07-07 {'avg': 0.1639526710402999}					
hori_sharebbn_s_05_ld_long_10434820_2024-02-07-12-07-07 {'avg': 0.37328425255802344}					

Table 2, Row 2

Single-Horizontal	Keep as is	Horizontal, Vertical	34.55	29.45	35.9
sharebbn_05_ld_long_10436205_2024-02-07-13-32-28 {'avg': 0.3454621600315333}					
sharebbn_05_ld_long_10436205_2024-02-07-13-32-28 {'avg': 0.29451733833177135}					
sharebbn_05_ld_long_10436205_2024-02-07-13-32-28 {'avg': 0.35902795108560015}					

Table 2, Row 3

Horizontal-MoE	Dropped	Short, Long	33.66	16.82	38.15
hori_sharebbn_62_ld_long_10434818_2024-02-07-12-06-42 {'avg': 0.3366180528182893}					
hori_sharebbn_62_ld_long_10434818_2024-02-07-12-06-42 {'avg': 0.16817010309278352}					
hori_sharebbn_62_ld_long_10434818_2024-02-07-12-06-42 {'avg': 0.381473047167457}					

Table 2, Row 4

MOoSE	Keep as is	Short, Long, Vertical	35.92	29.18	37.71
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.3591963933780055}					
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.2918228678537957}					
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.3771368854504617}					

Table 3, Row 1

Horizontal-MoE	Dropped	Short, Long	33.66
hori_sharebbn_62_ld_long_10434818_2024-02-07-12-06-42 {'avg': 0.3366180528182893}			

Table 3, Row 2

Rotated	Rotate to Horizontal	Short, Long	34.8
sharebbn_62RS_ld_long_10434822_2024-02-07-12-07-27 {'avg': 0.3479749704375246}			

Table 3, Row 3

Single-Horizontal	Keep as is	Horizontal, Vertical	34.55
sharebbn_05_ld_long_10436205_2024-02-07-13-32-28 {'avg': 0.3454621600315333}			

Table 3, Row 4

MOoSE	Keep as is	Short, Long, Vertical	35.92
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.3591963933780055}			

Table 4, Row 1

Share None			25.45	18.17	26.1
sharenone_62_ld_long_10452004_2024-02-09-08-38-33 {'avg': 0.24430922349231377}					
sharenone_62_ld_long_10452004_2024-02-09-08-38-33 {'avg': 0.18170103092783504}					
sharenone_62_ld_long_10452004_2024-02-09-08-38-33 {'avg': 0.26098078362864985}					

Table 4, Row 2

Share All	✓	✓	28.41	15.31	31.9
shareall_62_ld_long_10452003_2024-02-09-08-38-28 {'avg': 0.31901984027951086}					
shareall_62_ld_long_10452003_2024-02-09-08-38-28 {'avg': 0.2841323413480489}					
shareall_62_ld_long_10452003_2024-02-09-08-38-28 {'avg': 0.15311621368322398}					

Table 4, Row 3

MOOSE		✓	35.92	29.18	37.71
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.3591963933780055}					
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.2918228678537957}					
sharebbn_62_ld_long_10434821_2024-02-07-12-07-17 {'avg': 0.3771368854504617}					

Done.

For those who **believe** that **performances from different iterations subject to Normal Distribution**, you can meddle the code to enable the output of pair-wise t-tests.



Reproducing the Benchmarks

Great, now we want to check the remaining tables, and if our released models match the logs.

1. Download the models (40+Gib):

<https://www.kaggle.com/datasets/object300/model-moose-release>

2. The general approach:

a. Edit the test file (*eval_*.py) to choose the iteration you want to run from

```

Package requirements 'certifi==2023.11.17', 'MarkupSafe==2.1.3', 'numpy==1... Install requirements Ignore require

1 from neko_sdk.cfgtool.platform_cfg import neko_platform_cfg
2 import sys
3 from neko_sdk.neko_framework_NG.workspace import neko_environment
4
5 from loadout import get_test_mose_wandb,db_testing_protocol,anchor_setup,training_model_core
6
7 # modset.load("_E3_I0");
8 if __name__ == '__main__':
9     if(len(sys.argv)>1):
10         cfg=neko_platform_cfg(sys.argv[1]);
11     else:
12         cfg=neko_platform_cfg(None);
13     cfg.arm_wandb();
14     anchors=anchor_setup();
15     saveto, logto, testds=db_testing_protocol(cfg);
16     modset, _ = training_model_core(anchors, saveto, logto, trmeta: None);
17     modset.load("_E1");
18     modset.to(cfg.devices[0]);
19     modset.eval_mode();
20
21     tests=get_test_mose_wandb(anchors,cfg.run,testds);
22
23
24     e = neko_environment(modset=modset);
25     ta = tests["agent"]([tests["params"]]);
26     ta.take_action({}, e);

```

b. Run them.

3. Now let's verify the tables:

Table 5 (Note this will take a while as it will render results and dump them to your disk)

For the regular model, run `sharebbn_62_ld_long/eval_ostr_hv.py`

For the large model, run `yukon_sharebbn_62_ld_long_XL/eval_ostr_hv.py`

GZSL	Latin, Shared Kanji		MOOSE	34.27	-	-	-
JPNHV-JPNHV-GZSL_all ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.34272763106030746 ,ASNEED: 0.42176494282960925 ,Lenpred_ACR: 0.824398896334253 ,FPS: 24.212209338404254							
	Unique Kanji, Kana		MOOSE-XL	37.39	-	-	-
JPNHV-JPNHV-GZSL_all ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.3738667717776902 ,ASNEED: 0.37571522656115813 ,Lenpred_ACR: 0.8149389042175799 ,FPS: 28.545341704352165							
OSR	Latin, Shared Kanji	Unique Kanji	MOOSE	69.08	71.56	94.05	81.28
JPNHV-JPNHV-OSR_all' ,Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.6908249807247494 , 'R': 0.715647339158062 , 'P': 0.9405010438413361 , 'F': 0.812810103743798}							
	Kana		Moose-XL	75.56	74.05	95.79	83.53
'TEST': 'JPNHV-JPNHV-OSR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.7555898226676947 , 'R': 0.7405348159915277 , 'P': 0.9578767123287671 , 'F': 0.8352}							
GOSR	Latin, Shared Kanji	Kana	MOOSE	56.73	62.27	76.98	68.85
JPNHV-JPNHV-GOSR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.5673382820784729 , 'R': 0.6227171492204899 , 'P': 0.7698237885462555 , 'F': 0.68850036936715							
	Unique Kanji		Moose-XL	59.81	63.83	81.89	71.74
JPNHV-JPNHV-GOSR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.5980911983032874 , 'R': 0.6383073496659243 , 'P': 0.8188571428571428 , 'F': 0.717396}							
OSTR	Shared Kanji	Latin	MOOSE	59.96	80.42	84.91	82.61
JPNHV-JPNHV-OSTR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.5996751928542428 , 'R': 0.8042895442359249 , 'P': 0.8491710473109584 , 'F': 0.8261211644374508}							
	Unique Kanji	Kana	Moose-XL	61.75	80.01	88.18	83.9
JPNHV-JPNHV-OSTR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 5074.0 , 'KACR': 0.6175395858708892 , 'R': 0.8000765990042129 , 'P': 0.8818066694807936 , 'F': 0.838955}							

And if the fonts are too small try zooming it up. That's why PDFs are always better than paperbacks

Trivia: Yukon is the largest subspecies of the moose family.

Now, match these lines with our log

```
JPNHV-JPNHV-GZSL_all ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.34272763106030746 ,ASNE: 0.42176494282960925 ,Lenpred_ACR: 0.824398896334253 ,FPS: 24.212209338404254

Date: 2024-02-08 11:57:51.648944 ,TEST: JPNHV-gzsl_all ,Epoch: 1 ,Iter: 0 ,Total: 5074 ,ACR: 0.34272763106030746 ,ASNE: 0.4219409099448747 ,Lenpred_ACR: 0.824398896334253 ,FPS: 82.59956780138607
epoch done

JPNHV-JPNHV-GZSL_all ,Epoch: 0 ,Iter: 0 ,Total: 5074 ,ACR: 0.3738667717776902 ,ASNE: 0.37571522656115813 ,Lenpred_ACR: 0.8149389042175799 ,FPS: 28.545341704352165

20994 Date: 2024-02-08 21:38:27.550164 ,TEST: JPNHV-gzsl_all ,Epoch: 1 ,Iter: 0 ,Total: 5074 ,ACR: 0.3738667717776902 ,ASNE: 0.37571522656115813 ,Lenpred_ACR: 0.8149389042175799 ,FPS: 81.3376144798369
20995 epoch done
```

You can now test other models to verify if they match the logs we uploaded.

Table 6:

Run yukon_sharebbn_62_ld_long_XL/eval_ostr_g1.py

				MOoSE-XL*	39.56	-	-	-
TEST: JPN-JPN-GZSL_all ,Epoch: 0 ,Iter: 0 ,Total: 4009 ,ACR: 0.395609877750062 ,ASNE: 0.3425975596464096 ,Lenpred_ACR: 0.8116737340982789 ,FPS: 144								
				MOoSE-XL*	64.80	80.49	89.12	84.59
TEST: 'JPN-JPN-OSTR_all' , 'Epoch': 0 , 'Iter': 0 , 'Total': 4009.0 , 'KACR': 0.6480125523012552 , 'R': 0.804959465903672 , 'P': 0.8912354804646251 , 'F': 0.84590328238								

You may have noticed that single batch FPS reaches 144 now, fast, but if you want to meddle with the code you may reach faster results at multibatch.

Because it no longer renders individual results or writes to my cheap wearing-SSD.

HW: 4060Ti 16G on Thunderbolt3(TH3P4G3), 12400, 4*8GiB DDR5 @4000, Lenovo P360 Ultra.



Done. All numbers reported.



Testing Single images

Okay, now you are curious about our web pictures in Figure 1.

1. Grab the files from

<https://www.kaggle.com/datasets/object300/mose-extra?select=athenaNG>

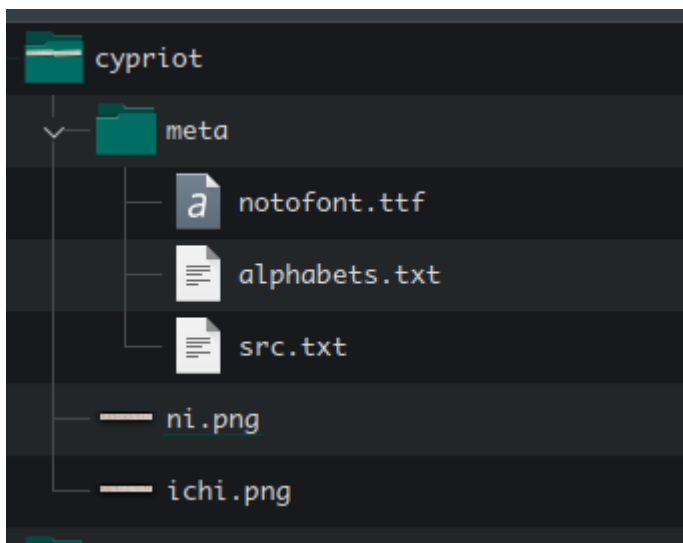
2. Set paths in yukon_sharebbn_62_ld_long_XL/eval_ostr_athena.py

```
SRC = "/home/lasercat/ssddata/athenaNG";  
DST = "/home/lasercat/ssddata/athenaNG_results";
```

2. Run yukon_sharebbn_62_ld_long_XL/eval_ostr_athena.py and collect results from DST

3. To add a language:

a. You need a folder like this

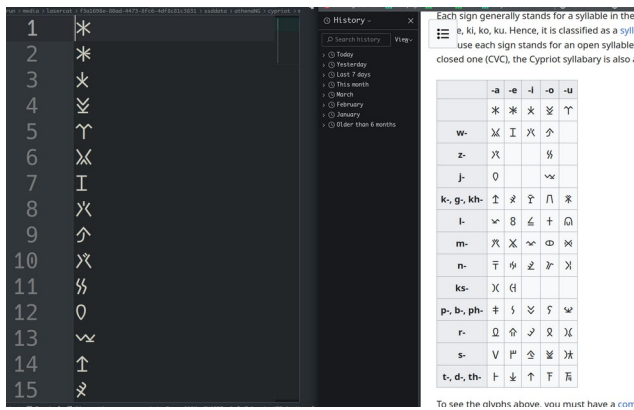


You first make a folder for the data and then you make a meta folder in it. For the meta folder

1) You download the corresponding noto font from google, and rename it to notofont.ttf

E.g. <https://fonts.google.com/noto/fonts?noto.query=Cypriots>

2) you scrape wikipedia to get the character set:



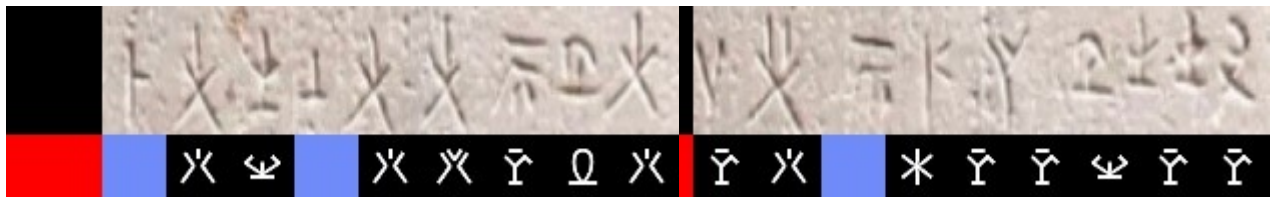
3) Then register it in your script:

yukon_sharebbn_62_ld_long_XL/eval_ostr_athena.py

```
langs = ["russian", "english", "korean", "cypriots"]
```

4) Finally, run the script and collect the results

(It will automatically generate the pt file if it's not there so don't worry)



Hit and miss, due to the carving style is not covered by the training set.

We will work on this in the future

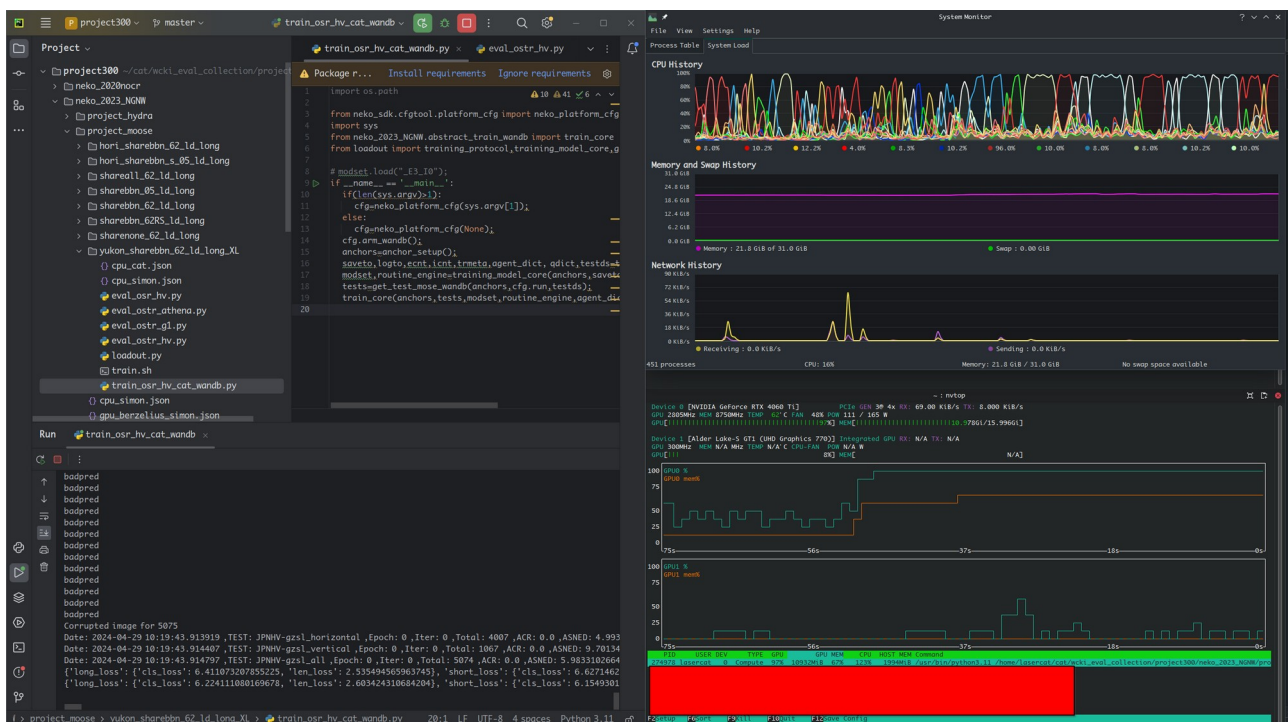
Why we are not using this to benchmark the model? I don't read or type Cypriots, nor do have enough data to benchmark the model with. Never bench a model with 10 samples.

It's not like I pick Japanese simply out of my personal preference. It's more or less a data availability issue.



Training your own model

Get your data from kaggle, and run
yukon_sharebbn_62_ld_long_XL/train_osr_hv_cat_wandb.py



If you see the “long_loss” lines, you are good.

The program takes 12 GB of FREE main memory, thus we recommend having 32GiB or more

You will roughly need 10-11 GB VRAM for training the large model.

We recommend Tesla P40s (server line) / 4060Ti (desktop line) or later models, though they are currently more expensive than they should be.

For extremely low-budget options, you may use M40-12GB, or if lucky it may run on a P102-10G GPU, which should be dirt cheap now.

WE DO NOT RECOMMEND THEM, but if you have to, you have to.

If anyone has P102s please confirm if that thing works or not.

If anyone has any luck on AMD GPUs, please let us know...

That's all for now.

This work only sets the stage, and we will be back with something far more interesting.

